

Snapchat Hacking and Masking - Final Advice

On behalf of the Rotterdam Police

Zoe J.D.M. Spyrou
Tim C.P. Moorsel

23/1/2026

Introduction.....	2
Context	3
1. Security research.....	3
Snapchat Web Assets Security Assessment	4
Scope and Objective	4
Method Overview	4
Key Findings	4
Overall Conclusion	5
Automated Web Reconnaissance Tool (script.py)	5
Scope and Objective	5
Method Overview	5
Key Strengths	5
Limitations and Opportunities.....	6
Overall Conclusion	6
2. LSPosed Module.....	6
Conclusion.....	7
3. Interface/GUI	7
Conclusion.....	7
4. Failed attempts	8
5. Risks and Limitations	8
Technical Risks	8
Android OS Updates.....	8
Operational Risks.....	8
Rooted Device Detection	9
Maintenance Risks	9
Module Ownership and Maintenance	9
6. Advice on Android devices.....	9
7. Final advice.....	9
8. References.....	10

Version Control

Date	Changelog
22/1/2026	First version of the document
23/1/2026	Clarification, section 1. Security Research, has been written by Tim Moorsel

Introduction

The police uses Snapchat to talk to suspected individuals, through fake/controlled accounts portraying to be underage children, using AI generated images. The goal of the project was to find a way to mask the activity of the police and extract the IP addresses of the suspects, so that they could then use that information to find out the location of the suspect.

Masking is defined as:

Make it appear as if messages sent on Snapchat are being sent on a mobile device, not a browser, as Snapchat notifies users if the person that contacted them is using the web version

Images and videos that are being sent need to appear as media that was just taken through Snapchat, and not as media directly uploaded through a camera roll, as those appear different in Snapchat, plus carry a “Media Upload” tag

Not notifying the suspect of screenshots taken of the conversation, needed for collection of evidence

All research and testing was done on accounts that were owned by us (the team that worked on the project). No non-consenting parties were used in testing/experimenting. These efforts are not to be used in any scope outside the project.

Context

During this project, multiple technical approaches were explored to determine whether Snapchat activity could be masked in a way that supports undercover police investigations. This paper summarizes the explored methods, including unsuccessful approaches, and explains why certain directions were abandoned while others proved viable.

1. Security research

Snapchat Web Assets Security Assessment

Scope and Objective

A controlled security assessment was performed on publicly accessible Snapchat web assets as part of an HBO-ICT project in cooperation with the Rotterdam police. The objective was to identify, validate, and assess potential security risks within exposed web infrastructure using responsible and non-invasive techniques.

Method Overview

Automated reconnaissance and vulnerability testing were conducted using a custom Python tool that combined subdomain enumeration, URL crawling, parameter analysis, and automated security checks. Findings were always manually validated to eliminate false positives and prevent misclassification.

Test areas included:

- Open redirect behavior
- Cross-Origin Resource Sharing (CORS) configuration
- Reflected input and potential XSS vectors
- JavaScript exposure and API endpoint visibility
- Directory exposure and SSRF behavior

Key Findings

Open Redirects

Parameters appeared vulnerable at first glance, but all tested redirects remained strictly within Snapchat's own domains. No external redirection or phishing potential was observed.

CORS Configuration

One static domain allowed wildcard origins. However, credentials were never shared and the content was strictly static. No sensitive data exposure or abuse potential was identified.

JavaScript and API Exposure

Several internal API endpoints were visible in client-side scripts, but no sensitive tokens or credentials were present. The observed header tokens were not exploitable outside their intended context.

XSS Indicators

Automated scanning flagged multiple potential XSS cases. Manual verification confirmed that all inputs were properly sanitized and not executable. These findings were false positives.

Other Findings

Exposed directory indicators and SSRF tests did not reveal any accessible data or internal resource exposure.

Overall Conclusion

No exploitable vulnerabilities were identified within the tested scope. All flagged items were neutralized by existing security controls such as input sanitization, strict redirect handling, HTTPS enforcement, and safe CORS configuration. The Snapchat web environment demonstrates a mature and resilient security posture at the time of testing.

It should be noted that security is dynamic. Future platform changes may introduce new attack surfaces, therefore periodic reassessment remains recommended.

Automated Web Reconnaissance Tool (script.py)

Scope and Objective

This evaluation focused on a custom-built Python script developed as part of an HBO-ICT project. The script aims to automate reconnaissance and basic web vulnerability scanning for security testing purposes. The goal was to assess its technical effectiveness, strengths, and areas for improvement in the context of ethical hacking and tool development.

Method Overview

The tool integrates well-known third-party utilities (e.g., Subfinder, GoSpider, Nuclei) to perform subdomain enumeration, URL crawling, and vulnerability checks such as XSS, SQLi, and SSRF. It operates through a menu-driven interface and outputs results in structured formats including HTML and JSON. Multithreading is used to improve performance, and each scan phase builds on the previous step in a pipeline-like approach.

Key Strengths

- Effective automation of reconnaissance tasks
- Integration of reliable open-source tools
- Parallel processing for faster scanning
- Professional-quality reporting formats

- Broad vulnerability coverage, including modern issues like CORS and SSRF

Limitations and Opportunities

- Code is written as a single large file, reducing maintainability
- Minimal error handling may lead to silent failures during scans
- Hardcoded tool logic limits flexibility and modularity
- No headless mode for automated or CI/CD use cases

Recommended improvements include refactoring the code into modules, improving error transparency, enabling advanced user configuration, and supporting non-interactive usage for broader adoption.

Overall Conclusion

The script offers solid value as a reconnaissance automation tool and educational project. It successfully consolidates essential scanning tasks into a usable workflow and demonstrates practical security testing concepts. With further refinement, it could evolve into a more maintainable and professional-grade solution suitable for both student and field use.

2. LSPosed Module

Following the limitations of web-based approaches, the focus shifted to the Android client itself. Snapchat distinguishes between media captured directly through the camera and media uploaded from the device's storage, marking the latter with a "media upload" tag.

Further research demonstrated that Snapchat relies on the Android camera pipeline rather than the file origin itself. By intercepting the camera stream at the framework level using an LSPosed module, it became possible to replace the camera output with pre-prepared media while still presenting it to Snapchat as live camera input.

A key technical discovery was that Snapchat expects individual JPEG image frames rather than a continuous video stream. Early implementations using MediaPlayer-based streaming failed, while a frame-by-frame approach successfully matched Snapchat's expectations. This resulted in media being accepted as freshly captured, without triggering upload indicators.

Conclusion

Camera interception at the Android framework level is an effective method for masking

media origin in Snapchat. However, this approach requires a rooted Android device, LSPosed, and strict control over device and application versions.

3. Interface/GUI

While the Android camera manipulation approach proved technically successful, it introduced significant usability challenges. Manually transferring files, interacting directly with a rooted phone, and validating output is impractical in a fast-paced investigative environment.

To address this, a desktop-based control and streaming solution was developed. Using scrcpy as a low-latency streaming backbone combined with a custom Python-based decoder and GUI, investigators can preview the device screen, control touch input, and manage media from a desktop interface.

This approach removes the need for constant physical interaction with the phone and significantly reduces operational complexity. The desktop interface acts as a control layer on top of the Android module, without modifying the underlying camera interception logic.

Conclusion

A desktop GUI combined with real-time streaming is essential for making the camera interception approach usable in practice. It enables faster operation, centralized control, and improved reliability during investigations.

4. Failed attempts

Before arriving at a viable solution, several approaches were explored that ultimately proved unsuitable for operational use. These failed attempts were an essential part of the research process, as they clarified Snapchat's detection mechanisms and helped define the constraints under which a successful solution would need to operate.

Trying to route media and messages from Snapchat Web to Snapchat (App) was not possible (in order to mask that the Web version was used), as the app's security prevented from any Man In The Middle attacks that would expose the data needed to complete this action

USB Camera/External camera usage was not possible. There was an assumption that a USB device inserted into the phone could act as an alternative camera for Snapchat to use. This led to the understanding of how the camera hardware of the Android phone actually interacts with Snapchat (and any other app), and the discovery of the Camera2 API

Running Snapchat in an Emulator was not possible either, or at least sustainable in the long run. Snapchat detects that the device being used is not a real phone/secure phone. This led to the understanding of device integrity and ways to mask tampered devices, which helped with the Magisk part of the final solution

5. Risks and Limitations

While the final solution works at the moment, it is subject to several risks in the future.

Technical Risks

Android OS Updates

The solution relies on a rooted Android device with Magisk and LSPosed. Android system updates may break compatibility with these components or change camera framework behavior, rendering the solution inoperable.

Mitigation:

Maintain devices on known-working Android versions and test updates on isolated devices before deployment.

Operational Risks

Rooted Device Detection

The use of rooted devices carries an inherent risk of detection, either by Snapchat or through future integrity checks. Detection could compromise investigations or result in account suspension.

Mitigation:

Use dedicated investigation devices with hardened configurations and limit their exposure to unnecessary applications or services.

Maintenance Risks

Module Ownership and Maintenance

The LSPosed module used is a modified fork of an existing project that is not actively maintained. Long-term reliability depends on continued maintenance.

Mitigation:

Document all modifications clearly.

6. Advice on Android devices

As mentioned in the Technical Guide as well, it is important to stick to a phone brand that is depicted in the following list: <https://github.com/zenfyrdev/bootloader-unlock-wall-of-shame>

Pay special attention the the sections “Proceed with caution” and “Safe for now”. Always check on Google first before setting on a model, if someone has made a guide yet for the specific device.

OnePlus was chosen by us for testing, as it is the simplest device to root. One of their latest phones is the OnePlus Nord CE5. This phone is around 240 euros and runs on Android 16, offering security updates at least until 2029. Rooting instructions for this specific phone are included in the Technical Guide.

7. Final advice

Based on the findings, web-based and emulated approaches are not suitable for masking Snapchat activity due to strong platform security controls. The only effective method identified is Android framework-level camera interception on real devices.

It is recommended to use dedicated investigation devices, avoid untested system updates, and maintain clear internal documentation of all tooling and modifications.

The desktop GUI is considered essential for practical use, as it improves efficiency and reduces operational risk.

Exploiting Snapchat's APIs and other weaknesses will open new possibilities, but will always come at the risk of the solution being patched as soon as it is released, or soon thereafter.

It is recommended that future research continues to be done on factors that interact with Snapchat, as opposed to being part of Snapchat.

Moreover, it is recommended to broaden the research on other social media platforms, such as Discord, TikTok and Instagram.

8. References

Utilize the "Reading Guide" uploaded, to view more information on any section mentioned, and more. Documentation in the reading guide is all the documentation we have worked on this semester, which analytically describes the process and research done on each section.